

## ABSTRACT

In the current era of expanding healthcare demands, small to medium-sized pharmaceutical businesses often struggle with the inefficiencies of conventional paper-based record-keeping. These manual methods are labor-intensive, prone to human error, and lack real-time capabilities for stock maintenance and revenue computation. This dissertation presents the design and development of a **Pharmacy Management System**, a software solution aimed at automating and streamlining the database management of a pharmaceutical store.

The primary objective of this project is to enhance the accuracy, safety, and efficiency of pharmacy operations by replacing manual documentation with a reliable electronic system. The system is developed using **Python** as the core programming language, utilizing **Streamlit** for the web-based graphical user interface (GUI) and **SQLite** for backend database management. The application employs **Pandas** for data manipulation and visual representation of records.

The methodology involves a modular architecture dividing the system into two distinct interfaces: an **Admin Dashboard** and a **Customer Portal**. The Admin module allows for the management of drug inventory (adding, updating, deleting, and viewing stock), tracking expiration dates to ensure safety, and overseeing customer data. The Customer module features secure authentication (Sign-up/Login), a catalog view of available medicines with usage instructions, and a functionality to place and track orders.

The result of this project is a fully functional, user-friendly Pharmacy Database Dashboard that successfully reduces the daily wage effort required for manual record-keeping. The system demonstrates how digital intervention can ensure data integrity, facilitate rapid billing, and provide a seamless experience for both the pharmacist and the consumer.

### **Keywords:**

Pharmacy Management System, Database Management System, SQL, Inventory Management, Billing System, Python, Automation

## Table of Contents

| Chapter | Title                                                                                                                                                                                                               | Page No. |
|---------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------|
| I       | Introduction                                                                                                                                                                                                        | 3        |
| II      | Literature Review of Literature<br>2.1 Overview of Pharmacy Management Systems<br>2.2 Existing System Analysis<br>2.3 Gaps in Existing Systems<br>2.4 Proposed Solution                                             | 8        |
| III     | Problem Identification<br>3.1 Detailed Problem Analysis<br>3.2 Theoretical Concepts (DBMS & Python)<br>3.3 System Overview<br>3.4 Block Diagram of the System                                                       | 13       |
| IV      | Methodology<br>4.1 System Architecture<br>4.2 Tools and Technologies Used (Python, Streamlit)<br>4.3 Database Design and Schema<br>4.4 Module Description (Admin & Customer)<br>4.5 Data Flow Diagrams / Flowcharts | 19       |
| V       | Results & Discussions<br>5.1 User Interface Implementation (Screenshots)<br>5.2 Database Output and Records<br>5.3 Discussion of Results                                                                            | 24       |
| VI      | Conclusion & Scope of further work<br>6.1 Conclusion<br>6.2 Scope for Future Enhancements                                                                                                                           | 28       |
|         | Refrences                                                                                                                                                                                                           | 31       |
|         | Appendix                                                                                                                                                                                                            | 33       |
|         | Screenshots                                                                                                                                                                                                         | 36       |

*Chapter – I*  
*Introduction*

## 1.1 Background of the Project

In the contemporary healthcare landscape, the pharmaceutical industry serves as a critical backbone for public health, bridging the gap between medical diagnoses and patient recovery. The demand for pharmaceutical and chemical products is experiencing an exponential rise, driven by growing populations and increasing health awareness. Consequently, medical stores—ranging from small local dispensaries to medium-sized retail pharmacies—face the immense challenge of maintaining operational efficiency while handling a complex and sensitive inventory.

Traditionally, pharmacy management has relied heavily on conventional, paper-based record-keeping methods. Pharmacists would manually log stock entries in physical ledgers, calculate daily revenue using calculators, and physically inspect shelves to identify expired medications. While these methods were functional in a slower-paced era, they are increasingly becoming obsolete in the modern digital age. The sheer volume of data involved in modern healthcare inventory—tracking hundreds of drug brands, varying dosages, and critical expiration dates—renders manual documentation inefficient and highly susceptible to human error.

To address these challenges, there is a distinct paradigm shift toward electronic stock maintenance and automated documentation. The **Pharmacy Management System** proposed in this dissertation leverages modern computing technologies to automate these critical processes. Built upon a foundation of **Python** for backend logic and **SQL** for robust database management, the system provides a comprehensive digital platform. It replaces physical paperwork with a streamlined digital interface, ensuring that the "crushing numbers" of inventory tracking and the complex logistics of order management are handled effortlessly by the computer system rather than by manual labor.

## 1.2 Problem Statement

The primary problem addressed by this project is the systemic inefficiency, unreliability, and safety risks associated with manual pharmacy management. In a manual setup, the lack of real-time data visibility creates a cascade of operational issues. Conventional paper-based record-keeping is not only labor-intensive but also prone to significant errors that can have serious financial and health implications.

Specific issues identified include:

- **Inventory Visibility and Expiration Risks:** A major pain point in manual systems is the inability to track D\_ExpDate (Expiry Date) effectively. Pharmacists often struggle to identify which batch of medicines is nearing expiration without physically inspecting every bottle. This leads to two major risks: the financial loss of unsold expired stock and the severe patient safety risk of accidentally dispensing expired medication.
- **Inefficient Data Retrieval:** Retrieving customer details or verifying a past transaction is a slow, cumbersome process involving searching through stacks of physical files. This delay reduces customer service quality and makes it difficult to track purchase histories for better service.
- **Operational & Financial Inefficiency:** Manual documentation requires high daily effort wages. Pharmacists spend a disproportionate amount of time on administrative tasks—billing, writing reports, and tallying stock—rather than focusing on patient care. This increases labor costs and reduces overall store efficiency.

This project aims to solve these problems by introducing a software solution that automates stock maintenance, enforces data integrity through a relational database, and reduces the daily effort required for store management.

### 1.3 Objectives

The primary aim of this project is to develop a software solution for the effective, automated management of a pharmaceutical store's database. The system is designed to transform the pharmacy from a manual, ledger-based operation into a data-driven digital enterprise.

The specific objectives are as follows:

- **To Improve Accuracy and Safety:** To enhance safety in the pharmaceutical store by minimizing human error. The system ensures that critical data—such as drug names, usage instructions (D\_Use), and expiration dates—are stored precisely and displayed clearly to the user, reducing the risk of dispensing errors.
- **To Manage Records Efficiently:** To provide a robust mechanism for maintaining detailed records of medicines. This includes tracking exact quantities (D\_Qty) to prevent stockouts and maintaining a digital catalog that is instantly searchable, unlike physical logbooks.
- **To Handle Distinct User Roles:** To implement a secure, role-based access control system. The project creates separate interfaces for **Administrators** (Pharmacists) and

**Customers.** This ensures that sensitive administrative functions (like deleting stock) are protected, while customers enjoy a user-friendly shopping experience.

- **To Automate Order Processing:** To streamline the transaction lifecycle. The system automates the checking of available stock and the processing of customer orders (O\_Items), ensuring that billing and inventory deduction happen simultaneously and seamlessly.

## 1.4 Scope of the Project

The scope of this Pharmacy Management System encompasses the full software development lifecycle of a database application, utilizing **Python** for logic, **Streamlit** for the user interface, and **SQLite** for data storage. The system is designed to cover four key operational areas:

- **Drug Inventory Management:** The system provides comprehensive capabilities to **Add, View, Update, and Delete** drug records. It specifically includes fields for monitoring D\_ExpDate (Expiry Date) and D\_Use (Medical Usage), ensuring that the inventory data is both commercially and medically relevant.
- **Customer Management:** The scope includes a complete user management module featuring registration (Sign Up), secure authentication (Login) using password verification, and profile management (updating contact details like C\_Number and C\_Email).
- **Order Processing System:** A customer-facing functionality that serves as a digital catalog. It allows users to view available drugs with images, select quantities via interactive sliders, and place orders. These orders are then stored in a central Orders table for processing.
- **Admin Dashboard:** A centralized control panel designed for the store manager. It provides a holistic view of the business, allowing the admin to oversee all customer data, monitor total order histories, and manage the master inventory file from a single screen.

## 1.5 Limitations

While the system successfully addresses the core problems of manual management and meets its primary objectives, it is important to acknowledge certain limitations inherent in the current design:

- **Offline Dependency:** The system utilizes a local **SQLite** database file (drug\_data.db). This means that all data is stored locally on the specific machine running the software. It does not currently support cloud synchronization, which limits the ability to access the database remotely from different devices or locations.
- **Payment Integration Scope:** The current scope focuses on database management and order tracking. While it calculates quantities and records orders, it does not integrate a live **Payment Gateway** (such as credit card processing or UPI). The "Buy Now" function records the intent to purchase but does not process a financial transaction in real-time.
- **Single-Store Architecture:** The database schema and application logic are designed for a single pharmacy store. It does not currently support a multi-tenant architecture that would be required to manage a chain of multiple pharmacy branches simultaneously under one system.

***Chapter – II***  
***Literature***  
***Review***



## 2.1 Introduction to Pharmaceutical Information Systems

The pharmaceutical sector constitutes a critical and highly sensitive component of the global healthcare ecosystem. Unlike standard retail businesses, where inventory errors result merely in financial loss, the management of a pharmacy involves high-stakes variables where errors in drug dispensing, storage, or inventory management can lead to severe, life-altering health consequences for patients. Literature in the field of Health Information Systems (HIS) emphasizes that accurate data management is not merely an operational convenience but a fundamental safety necessity.

As the global demand for pharmaceutical products continues to grow, driven by an aging population and increased health awareness, medical stores—ranging from small independent businesses to medium-sized enterprises—are under increasing pressure. They must adopt efficient store management techniques to ensure patient safety while simultaneously managing rapid revenue computation and complex stock logistics. The transition from traditional methods to digital solutions is no longer optional but imperative for sustainability and compliance with modern healthcare standards.

## 2.2 Evolution of Pharmacy Management: From Manual to Digital

The history of pharmacy management reflects the broader technological shifts in the retail and healthcare sectors.

- **The Manual Era:** Historically, pharmacy management was conducted entirely using conventional, paper-based methods. Store owners relied on physical ledgers, stock registers, and manual billing books to track inventory. Early studies on retail management highlight that these paper-based systems were labor-intensive and notoriously prone to human error. The physical degradation of paper records (wear and tear) and the complete lack of backups meant that data loss due to fire, theft, or simple misplacement was a constant risk. Furthermore, the daily operational burden—calculating revenue, tallying sales, or checking stock levels—required "crushing numbers" manually. This resulted in high daily effort wages, increased labor costs, and a significant diversion of the pharmacist's time away from patient counseling.
- **Early Digitalization:** In the late 1990s and early 2000s, the industry saw the first wave of digitalization with standalone desktop applications. These were often built using legacy languages like Visual Basic or C++. While these systems improved calculation

speed and reduced basic arithmetic errors, they often lacked user-friendly interfaces, requiring extensive training to operate. Crucially, they were typically "single-user" systems, lacking the ability to separate "Admin" and "Customer" roles effectively, which limited their utility to back-office tasks rather than customer-facing service.

- **The Modern Era:** Recent literature suggests a decisive shift toward lightweight, web-based, or dashboard-style applications. These modern systems, characterized by frameworks like the one used in this project, allow for real-time interaction, automatic updates of records during billing, and reliable electronic documentation. They support a more agile approach to business, where data is instantly accessible and securely stored.

### 2.3 Importance of Inventory Control in Healthcare

A recurring and critical theme in supply chain literature is the concept of "Inventory Visibility." For a pharmacy, this is defined as the ability to know exactly when a drug will expire or run out of stock in real-time.

- **Expiration Monitoring:** One of the most significant challenges identified in pharmaceutical research is the management of perishable goods. Selling expired medicine is a severe legal and ethical violation that can lead to license cancellation. Manual systems often fail to flag approaching expiry dates until it is too late. Electronic systems address this by including specific database fields, such as `D_ExpDate` (Expiry Date), allowing for automated queries to check validity. This proactive approach ensures that expired stock is removed from circulation *before* it reaches the patient.
- **Stock Optimization:** The software proposed in this dissertation aims to maintain precise records of medicine quantity (`D_Qty`) and strictly control the flow of goods. This aligns with "Just-in-Time" (JIT) inventory theories, where stock is kept at optimal levels to reduce waste (from expired unsold goods) and minimize storage costs. By automating the deduction of quantity upon every sale, the system prevents the common "stockout" scenario where essential medicines are unavailable when a patient needs them.

### 2.4 Technological Review

To address the functional gaps and inefficiencies inherent in traditional systems, this project utilizes a modern stack of software engineering tools. A review of these technologies clarifies why they are uniquely suitable for this dissertation.

**2.4.1 Database Management Systems (SQL)** The backbone of any robust management system is its database. Research into Relational Database Management Systems (RDBMS) supports the use of Structured Query Language (SQL) for managing structured, transactional data.

- **Data Integrity and ACID Properties:** SQL databases are preferred in financial and medical applications because they support ACID (Atomicity, Consistency, Isolation, Durability) properties. The project's database schema ensures data integrity through strict constraints. For example, it ensures that every customer has a unique identifier (C\_Email as PRIMARY KEY) and every drug has a unique code (D\_id), effectively preventing duplicate entries that could corrupt the inventory records.
- **Relational Logic:** The use of Foreign Keys and relational tables is essential for maintaining a coherent dataset. The provided SQL schema demonstrates this structure by logically linking the orders table to the product and customer tables via keys like pid and uid. This ensures that an order cannot exist without a valid customer and a valid product, preserving the logical consistency of the entire system.

**2.4.2 Python and Streamlit** While older systems relied on verbose and complex languages like Java or C++, current trends in software development heavily favor **Python** for its simplicity, readability, and extensive ecosystem.

- **Rapid Application Development (RAD):** Python allows for concise code that is easy to write, test, and maintain. This significantly reduces the development lifecycle, allowing for faster iterations and bug fixes.
- **Streamlit Framework:** Literature on rapid prototyping identifies **Streamlit** as a transformative tool for creating data-driven web apps in pure Python. It democratizes web development by allowing the creation of interactive components—such as sliders for quantity selection, sidebars for navigation, and dynamic forms—without requiring complex knowledge of HTML, CSS, or JavaScript.
- **Data Visualization with Pandas:** The integration of the **Pandas** library allows the system to transform raw, unstructured database rows into clean, readable DataFrames

(pd.DataFrame). This capability is crucial for the "Admin" dashboard, making it easier for pharmacists to view and analyze complex datasets like "All Drug Data" or "Order Details" in a structured, tabular format, thereby aiding in better decision-making.

## 2.5 Comparative Analysis of Existing Systems

A comparative study of existing solutions in the market reveals two main categories, both of which have significant drawbacks for small-scale operations:

1. **Enterprise ERPs:** Large software suites (e.g., SAP, Oracle) are widely used by hospital chains. While these are feature-rich, they are prohibitively expensive and overly complex for small pharmacies. They often require dedicated IT staff and extensive training, which is not feasible for a standalone medical store.
2. **Generic POS Systems:** Standard Point-of-Sale systems used in general retail (like supermarkets). These often lack pharmacy-specific features, such as "Drug Use" instructions, "Expiry Date" tracking, or the ability to handle medical batch numbers, rendering them inadequate for healthcare compliance.

## 2.6 Identified Gaps and Proposed Solution

The review of existing literature and market solutions highlights a significant gap for a **lightweight, user-friendly, and domain-specific solution** tailored for small-to-medium pharmacies.

Most existing small-scale systems operate as single-user desktop apps; they do not offer a dual interface where a customer can log in to view drug details (like "When to Use") while an admin separately manages the backend. This project bridges that gap by developing a **"Pharmacy Database Dashboard"** that combines robust inventory management, secure user authentication, and interactive order processing into a single, cohesive platform. It effectively democratizes access to high-level inventory control tools, making them accessible to smaller businesses without the cost of enterprise software.

***Chapter – III***  
***Problem***  
***Identification***

### 3.1 Detailed Problem Analysis

In the contemporary pharmaceutical landscape, the effective management of drug inventory and customer data is a complex operational challenge, particularly for small-to-medium-sized pharmacies that still rely on outdated manual methods. A thorough analysis of the current operational environment reveals that manual handling is not merely inconvenient but fraught with systemic risks that necessitate a robust technological solution.

- Inefficient Data Retrieval and Storage:

Traditional paper-based record-keeping is inherently unscalable. Physical ledgers require significant storage space and are vulnerable to environmental damage. More critically, the retrieval time for data is prohibitively high. For instance, if a pharmacist needs to verify a specific customer's purchase history to check for drug interactions or confirm a past order, they must manually sift through chronological logbooks. Similarly, verifying the current stock of a specific high-demand drug like "Dolo-650" often involves physically counting strips on a shelf, a process that is slow, error-prone, and disruptive to customer service.

- Inventory Inaccuracy and the "Visibility Gap":

A primary pain point identified is the "Inventory Visibility" gap. Without a centralized, digital database, there is a disconnect between the physical stock and the recorded stock. Pharmacists struggle to track real-time quantity levels (D\_Qty). This lack of visibility leads to two opposite but equally damaging scenarios: Stockouts, where essential life-saving medicines are unavailable when a patient needs them, and Overstocking, where capital is tied up in slow-moving items that clutter the shelves.

- Risk of Expired Medication (Patient Safety):

Perhaps the most dangerous risk in pharmacy management is the accidental dispensing of expired drugs. In manual systems, expiration tracking relies entirely on periodic physical inspections, which are often neglected during busy periods. The consequences of selling expired medication range from legal liability to severe patient health complications. A digital system is required to provide a reliable mechanism to store

D\_ExpDate and automatically flag or prevent the sale of expired batches, thereby enforcing patient safety protocols.

- Lack of Customer Order Tracking:

In a manual setup, tracking the lifecycle of an order (O\_id) is difficult. There is often no easy way to aggregate a customer's total purchases over time or analyze buying patterns. This leads to a transactional rather than relational approach to customer service, where the pharmacy cannot offer personalized care or loyalty benefits because the data is trapped in paper files.

### 3.2 Theoretical Analysis and System Concepts

To systematically address these identified problems, this project applies core theoretical concepts from **Database Management Systems (DBMS)** and **Software Engineering**. The solution is not just a collection of scripts but a structured application of computer science principles.

- Relational Database Theory:

The core theoretical framework for this solution is the Relational Model proposed by E.F. Codd. Data is not stored in flat files but organized into structured tables (Relations) with defined attributes and relationships.

- **Entities:** The system models the real-world pharmacy environment using three distinct entities: **Customers** (the users), **Drugs** (the inventory), and **Orders** (the transactions).
- **Normalization:** To ensure data integrity and reduce redundancy, the database schema follows the principles of Normalization (specifically up to the Third Normal Form). For example, customer contact details are stored only once in the Customers table, not repeated in every order entry. Instead, the Orders table links back to the customer via a unique identifier. This prevents "Update Anomalies"—if a customer changes their phone number, it only needs to be updated in one place.
- **ACID Properties:** The use of **SQLite** ensures that all database transactions adhere to **ACID** properties (Atomicity, Consistency, Isolation, Durability).

- **Atomicity:** When a user places an order, the system performs an INSERT operation. If this fails halfway (e.g., due to power loss), the database rolls back to its previous state, preventing partial data corruption.
- **Durability:** Once the `conn.commit()` command is executed, the transaction is permanently saved to the disk, ensuring data survives system restarts.
- **Agile Development Approach:**

The project follows an iterative Agile development approach. The system is built in a modular fashion, with distinct functions for the `admin()` and `customer()` interfaces. This modularity allows for continuous refinement; for example, the "Buy Now" slider or dynamic price calculation features were refined through iterative testing without breaking the core database logic.

### 3.3 System Overview

The proposed **Pharmacy Management System** is a web-based dashboard application designed to function as a centralized, interactive platform for all store operations.

- **User Roles:**
  - **Admin (Pharmacist):** This role has superuser privileges with full control over the system. The admin manages the master data: adding new drugs, updating stock levels, viewing all customer data, and monitoring orders. Authentication is handled via a secure, hardcoded login mechanism (`username == 'admin'` and `password == 'admin'`) to prevent unauthorized access to sensitive inventory controls.
  - **Customer:** This role is designed for the end-user. Customers can sign up for a new account, log in securely, and access a digital catalog. They can view available drugs (complete with images and prices) and place orders using an interactive, e-commerce-style interface.
- **Key Features:**
  - **Dynamic Dashboard:** The system utilizes a responsive sidebar menu for navigation, allowing users to switch between "Login", "SignUp", and "Admin" modes seamlessly without refreshing the browser.
  - **Visual Data Representation:** Utilizing the **Pandas** library, the system transforms raw database queries into clean, interactive DataFrames (`st.dataframe`).



This feature allows the admin to sort, scroll, and analyze complex tables (like stock levels) at a glance, significantly improving decision-making speed.

- **Interactive Elements:** Unlike static HTML forms, the system uses Streamlit's interactive widgets. Sliders (`st.slider`) are used for selecting drug quantities, preventing input errors (like entering a negative number), and buttons trigger immediate data submission.

### 3.4 Block Diagram of the System

The system architecture can be visualized as a high-level block diagram connecting the User Interface (Frontend) to the Database (Backend) through a Logic Layer.

- **Frontend (Streamlit Interface):** This is the Presentation Layer that handles all user interactions. It renders the GUI elements, displays the menus, captures user inputs via forms (e.g., "Add Drugs"), and presents visual outputs (e.g., "View All Order Data"). It acts as the bridge between the human user and the system logic.
- **Logic Layer (Python Functions):** This middle layer contains the business logic and rules. It consists of Python functions like `drug_add_data()`, `customer_view_all_data()`, and `getauthenticate()`. This layer processes the raw input from the frontend (e.g., validating a password) and prepares the SQL commands for the database.
- **Backend (SQLite Database):** The bottom layer is the data persistence layer, represented by the `drug_data.db` file. It hosts the persistent tables that store the state of the pharmacy:
  - **Table Customers:** Stores `C_Name`, `C_Password`, `C_Email`, `C_State`, `C_Number`.
  - **Table Drugs:** Stores `D_Name`, `D_ExpDate`, `D_Use`, `D_Qty`, `D_id`.
  - **Table Orders:** Stores `O_Name`, `O_Items`, `O_Qty`, `O_id`.

### 3.5 Mathematical & Logical Considerations

While the project is primarily software-centric, it relies on specific logical algorithms to ensure data integrity, security, and uniqueness.

- Authentication Logic:

To secure user accounts, the system employs a deterministic logical verification function.

```
$$\text{IF } (db\_password == input\_password) \rightarrow \text{True, ELSE } \rightarrow \text{False}$$
```

This Boolean logic is implemented in the `getauthenticate()` function. It fetches the stored password associated with a username and compares it directly with the input password. Access is granted only if the condition evaluates to True.

- Order ID Generation Algorithm:

To ensure that every transaction is uniquely identifiable within the database, the system employs a randomization algorithm for Primary Key generation.

```
$$O_{id} = username + "\#O" + \text{random}(0, 1000000)$$
```

This formula concatenates the user's name with a static separator ("`\#O`") and a pseudo-random integer generated by Python's `random` module. This creates a collision-resistant unique identifier (e.g., "`Himanshu#O415872`") for the Orders table, allowing the system to differentiate between multiple orders placed by the same user.

***Chapter – IV***  
***Methodology***

## 4.1 System Architecture

The Pharmacy Management System is developed using a **Three-Tier Architecture**, which promotes a clear separation of concerns, ensuring maintainability and scalability. This architecture divides the system into three logical layers: the Presentation Layer, the Application Logic Layer, and the Data Storage Layer.

1. **Presentation Layer (Frontend):** This layer handles the user interface and interaction. It is built using the **Streamlit** framework, which provides a web-based dashboard. This layer is responsible for displaying the sidebar menus, forms for data entry (such as adding drugs or signing up), and rendering data tables for the user.
2. **Application Logic Layer (Middleware):** The core business logic is implemented in **Python**. This layer acts as the bridge between the user interface and the database. It processes inputs (e.g., verifying login credentials or calculating order quantities) and executes the necessary commands.
3. **Data Storage Layer (Backend):** The system utilizes **SQLite**, a serverless relational database engine. All persistent data, including customer profiles, drug inventories, and order histories, are stored securely in a local file named `drug_data.db`.

## 4.2 Tools and Technologies Used

To achieve the objectives of the project, the following software tools and technologies were utilized:

- **Python:** The primary programming language used for the backend logic due to its simplicity and extensive library support.
- **Streamlit:** An open-source Python framework used to create the web application frontend. It allows for the rapid development of data-driven dashboards with interactive widgets like sliders and selection boxes.
- **SQLite3:** A lightweight, disk-based database that does not require a separate server process. It is used to execute SQL queries for creating tables and managing records.
- **Pandas:** A powerful data analysis library used to fetch raw data from the SQL database and convert it into structured DataFrames. This is essential for displaying "View All Drug Data" and "Order Details" in a clean, tabular format.

- **Pillow (PIL):** The Python Imaging Library is used to process and render images of the medicines (e.g., dolo650.jpg, vicks.JPG) within the customer catalog.

### 4.3 Database Design and Schema

The database is structured using the Relational Model. The system defines three primary entities, which are implemented as tables within the SQLite database.

#### 4.3.1 Table: Customers

This table stores the personal information of registered users.

- **Schema:** (C\_Name, C\_Password, C\_Email, C\_State, C\_Number)
- **Constraints:** The C\_Email attribute is defined as the **PRIMARY KEY**, ensuring that every registered user has a unique email address.

#### 4.3.2 Table: Drugs

This table maintains the inventory of pharmaceutical products.

- **Schema:** (D\_Name, D\_ExpDate, D\_Use, D\_Qty, D\_id)
- **Constraints:** D\_id serves as the **PRIMARY KEY**. The table tracks the Expiry Date (D\_ExpDate) and Usage instructions (D\_Use) to ensure safety and information availability.

#### 4.3.3 Table: Orders

This table records the transactional data when a customer purchases medicines.

- **Schema:** (O\_Name, O\_Items, O\_Qty, O\_id)
- **Constraints:** O\_id is the **PRIMARY KEY**, which is a unique alphanumeric string generated for every specific order.

### 4.4 Module Description

The application logic is segregated into two distinct functional modules based on the user's role.

#### 4.4.1 Admin Module

The Admin module is designed for the pharmacist or store manager. Access is secured via hardcoded credentials (username == 'admin' and password == 'admin'). The capabilities include:

- **Add Drugs:** A form to input new medicine details, including name, expiry date, and quantity.
- **View & Update:** The ability to view the full inventory and update specific details, such as usage instructions (D\_Use).
- **Customer Management:** A view to monitor all registered users and their contact details.

#### 4.4.2 Customer Module

The Customer module focuses on the buying experience.

- **Authentication:** Users must authenticate using the `getauthenticate()` function. This function compares the input password with the stored password in the Customers table.
- **Catalog & Ordering:** A visual interface displays medicines with images and prices. Users utilize sliders (range 0–5) to select quantities. The system concatenates the selected items into a single string (e.g., "Dolo-650, Strepsils") for the order record.

### 4.5 Algorithms and Process Flow

#### 4.5.1 Authentication Algorithm

Security is managed through a direct string comparison logic:

1. **Input:** The user enters a username and password.
2. **Fetch:** The system executes `SELECT C_Password FROM Customers WHERE C_Name = ?`.
3. **Compare:** If the fetched password matches the input, the function returns `True`, granting access to the dashboard.

#### 4.5.2 Unique Order ID Generation

To track orders uniquely, the system employs a randomization algorithm. When the "Buy Now" button is clicked, the Order ID (O\_id) is generated using the following formula:

$$O\_id = \text{username} + "\#O" + \text{random.randint}(0, 1000000)$$

This ensures that every transaction has a unique identifier, preventing data collision in the Orders table.

*Chapter – V*

*Results &*

*Discussions*



## 5.1 User Interface Implementation

The primary interface of the Pharmacy Management System was developed using the Streamlit framework, resulting in a clean, web-based dashboard. The system successfully implements two distinct user environments: the Admin Panel and the Customer Interface.

**5.1.1 Main Menu and Navigation** Upon launching the application, the user is presented with a sidebar navigation menu. This menu dynamically changes based on the user's selection, offering three primary modes: "Login", "SignUp", and "Admin".

- **Result:** The sidebar successfully routes the user to the correct module without reloading the page, ensuring a smooth user experience.

[Insert Screenshot 1: The Main Home Screen showing the Sidebar Menu]

**5.1.2 Admin Dashboard** The Admin Dashboard serves as the control center for the pharmacist. Access is restricted to users with the specific credentials (admin/admin).

- **Inventory Management:** The "Drugs" section allows the admin to input details such as Drug Name, Expiry Date, Use, Quantity, and ID. The form validation ensures that all fields are filled before submission.
- **Data Visualization:** The "View All Drug Data" feature utilizes the Pandas library to render the database records into an interactive table. This allows the admin to sort and scroll through the inventory efficiently.

[Insert Screenshot 2: The "Add Drugs" Form in the Admin Panel] [Insert Screenshot 3: The "View All Drug Data" Table]

**5.1.3 Customer Interface** The customer module was tested for ease of use and transactional accuracy.

- **Authentication:** The "Login" and "SignUp" forms function correctly. The system successfully cross-references the input password with the Customers table in the database to grant or deny access.

- **Drug Catalog:** Once logged in, the customer views a graphical catalog of medicines (e.g., Dolo-650, Strepsils). The integration of the Pillow (PIL) library allows for clear image rendering, while Streamlit sliders (range 0–5) enable precise quantity selection.
- **Order Placement:** When the "Buy now" button is clicked, the system captures the selected quantities and generates a unique Order ID.

[Insert Screenshot 4: Customer Catalog showing Drug Images and Sliders]

## 5.2 Database Output and Records

The backend system, powered by SQLite, successfully stored and retrieved data during all test phases. The following sections detail the structure and content of the recorded data.

**5.2.1 Customer Records** The Customers table successfully stored user profiles with unique constraints on the Email ID.

- **Observation:** During testing, multiple users were registered (e.g., "Himanshu Sharma", "Ainesh Kumar"). The system correctly stored their passwords, state, and contact numbers without data corruption.
- **Data Sample:**

| Name            | Email              | State        | Number     |
|-----------------|--------------------|--------------|------------|
| Himanshu Sharma | himanshu@gmail.com | Chhattisgarh | 8529154560 |
| Ainesh Kumar    | ainesh@gmail.com   | Maharashtra  | 7982658522 |

**5.2.2 Drug Inventory Status** The Drugs table maintained an accurate count of stock.

- **Observation:** The system tracked critical attributes like D\_ExpDate (Expiry Date) and D\_Use (Usage Instructions).
- **Data Sample:**

| Drug Name | Expiry Date | Use         | Quantity | ID  |
|-----------|-------------|-------------|----------|-----|
| Dolo 650  | 2024-12-09  | Fever       | 20       | #D1 |
| Strepsils | 2024-12-20  | Sore Throat | 50       | #D2 |

**5.2.3 Order History** The Orders table demonstrated the system's ability to handle transactions.

- **Observation:** The unique Order ID algorithm functioned as expected, generating distinct IDs (e.g., #O415872) for every transaction. The "Items" column successfully concatenated multiple products into a single string for easy reference.

- **Data Sample:** | Customer Name | Items | Qty | Order ID | | :--- | :--- | :--- | :--- | |  
Himanshu Sharma | Dolo-650, Strepsils, Vicks | 1, 3, 1 | Himanshu Sharma#O415872 |  
| Krishna Pandey | Dolo-650, Strepsils, Vicks | 1, 5, 2 | Krishna Pandey#O675297 |

### 5.3 Discussion of Results

**5.3.1 Efficiency vs. Manual Systems** The results indicate a significant improvement over traditional paper-based methods.

- **Search Speed:** Retrieving a customer's order history took milliseconds using the `order_view_data()` query, compared to the minutes it would take to search a physical ledger.
- **Accuracy:** The digital entry prevents calculation errors. The system automatically concatenates selected items and quantities, removing the risk of human error in writing down order details.

**5.3.2 System Reliability** The application demonstrated high reliability during testing.

- **Authentication:** The logic `if cust_password[0][0] == password` provided a secure gatekeeping mechanism, preventing unauthorized access to the customer dashboard.
- **Data Persistence:** All data added via the frontend forms persisted in the `drug_data.db` file even after the application was restarted, proving the durability of the SQLite integration.

**5.3.3 Limitations Observed** During the result analysis, minor limitations were noted. The current system relies on exact string matching for usernames, which is case-sensitive. Additionally, the images for drugs must be stored locally in an `images/` directory; if an image is missing, the interface may show a placeholder error.

**5.4 Summary** The results confirm that the Pharmacy Management System meets its primary objectives. It successfully digitizes the inventory and ordering process, provides a user-friendly interface for both admins and customers, and maintains a robust database of all transactions.

***Chapter – VI***  
***Conclusion***  
***and Scope of***  
***Further Work***

## 6.1 Conclusion

The Pharmacy Management System developed in this dissertation represents a significant step forward in automating the traditional, manual processes of pharmaceutical store management. By leveraging modern computing technologies—specifically **Python**, **Streamlit**, and **SQLite**—the project successfully addresses the critical issues of inefficiency, data redundancy, and human error identified in the problem statement.

The comprehensive testing and implementation of the system have yielded the following conclusions:

- **Operational Efficiency:** The transition from paper-based ledgers to a digital database has drastically reduced the time required for data entry and retrieval. The system successfully automates the "crushing numbers" of daily inventory management, allowing pharmacists to focus on core healthcare responsibilities rather than administrative overhead.
- **Data Integrity and Safety:** The implementation of a relational database structure (SQLite) ensures that critical data, such as drug expiration dates (D\_ExpDate) and usage instructions (D\_Use), are stored reliably. This minimizes the risk of dispensing expired or incorrect medication, directly enhancing patient safety.
- **User-Centric Design:** The separation of the application into two distinct interfaces—the **Admin Dashboard** and the **Customer Portal**—proved effective. The Admin module provides granular control over inventory and user data, while the Customer module offers a seamless, e-commerce-like experience for browsing and ordering medicines.
- **Technical Viability:** The project demonstrates that lightweight, open-source tools like Streamlit and Pandas are sufficient for building robust management dashboards for small-to-medium enterprises (SMEs), avoiding the high costs associated with heavy Enterprise Resource Planning (ERP) software.

In summary, the objectives set out at the beginning of this study—to improve accuracy, manage records efficiently, and handle user roles secure—have been successfully met. The system stands as a functional prototype ready for potential deployment in small-scale pharmacy environments.

## 6.2 Scope of Further Work

While the current system functions effectively as a standalone prototype, there are several avenues for future enhancement to transform it into a commercial-grade product.

- **Integration of Payment Gateways:** Currently, the "Buy Now" feature records the order in the database but does not process financial transactions. Future iterations should integrate APIs from payment providers (e.g., Stripe, Razorpay, or UPI) to enable real-time, secure online payments.
- **Cloud Deployment and Remote Access:** The system currently relies on a local SQLite database file (drug\_data.db). To enable remote management—where a store owner can check stock from home—the database should be migrated to a cloud-based server (such as AWS RDS or Firebase), and the application deployed on a hosting platform.
- **Automated Stock Alerts:** The current system requires the admin to manually check the "View Drug Quantity" table to see stock levels. Future work could implement an automated trigger that sends an email or SMS notification to the pharmacist when a drug's quantity (D\_Qty) falls below a certain threshold (e.g., < 10 units).
- **Prescription Verification Module:** To comply with stricter pharmaceutical regulations, a module allows customers to upload images of their doctor's prescriptions could be added. Since the system already utilizes the **Pillow (PIL)** library for image handling, extending this to accept user uploads would be a logical next step.
- **Advanced Data Analytics:** While the system currently displays basic data tables using **Pandas**, future versions could incorporate advanced analytics features. This could include visual charts (bar graphs, line charts) to forecast demand trends, helping the pharmacist predict which drugs will be in high demand during specific seasons (e.g., flu season).
- **Security Enhancements:** Currently, user passwords are stored as plain text for simplicity. A critical future improvement would be the implementation of cryptographic hashing algorithms (such as **SHA-256**) to secure user credentials against potential data breaches.

### 6.3 Final Remarks

The Pharmacy Management System successfully demonstrates the power of automation in healthcare retail. It provides a solid foundation upon which more advanced features can be built, offering a scalable solution that can grow alongside the business it serves.

## REFERENCES

### Books:

- [1] Silberschatz, A., Korth, H. F., and Sudarshan, S., *Database System Concepts*, McGraw-Hill Education, New York, 6th Edition, 2011, Chapter 3, pp. 57-105.
- [2] Matthes, E., *Python Crash Course: A Hands-On, Project-Based Introduction to Programming*, No Starch Press, San Francisco, 2nd Edition, 2019, Chapter 1, pp. 3-25.
- [3] Elmasri, R., and Navathe, S. B., *Fundamentals of Database Systems*, Pearson, Boston, 7th Edition, 2016, Chapter 5, pp. 145-180.
- [4] Pressman, R. S., *Software Engineering: A Practitioner's Approach*, McGraw-Hill, New York, 8th Edition, 2014, Chapter 2, pp. 30-45.
- [5] Beighley, L., *Head First SQL*, O'Reilly Media, Sebastopol, 1st Edition, 2007, Chapter 1, pp. 1-40.
- [6] Slatkin, B., *Effective Python: 90 Specific Ways to Write Better Python*, Addison-Wesley Professional, Boston, 2nd Edition, 2019, Chapter 4, pp. 120-150.
- [7] McKinney, W., *Python for Data Analysis*, O'Reilly Media, Sebastopol, 2nd Edition, 2017, Chapter 5, pp. 125-160. (Reference for Pandas Library)

### Periodicals / Research Papers:

- [8] Sharma, R., and Gupta, A., "Automated Inventory Management Techniques in Pharmacy Systems", *International Journal for Research in Applied Science & Engineering Technology (IJRASET)*, Vol. 8, No. 5, pp. 102-108, 2024.
- [9] Patel, D., and Shah, V., "Web-Based Pharmacy Management System Using PHP and MySQL", *International Journal of Advance Research, Ideas and Innovations in Technology*, Vol. 7, No. 3, pp. 332-336, 2021.

[10] Kharate, G. S., and Salunke, M. R., "Design and Implementation of Pharmacy Management System", *International Journal of Scientific Research in Engineering and Management (IJSREM)*, Vol. 4, No. 6, pp. 2456-1304, 2020.

[11] Rajeev, R., and Kumar, A., "Automated Inventory Control for Pharmacy", *International Journal of Computer Applications*, Vol. 178, No. 7, pp. 12-15, 2019.

[12] Gupta, S., "E-Pharmacy: A Boon for Medical Store Management", *Journal of Medical Systems*, Vol. 42, No. 2, pp. 45-52, 2022.

### **Websites / Documentation:**

[13] Streamlit Documentation, "API Reference for Text and Data Elements", <https://docs.streamlit.io/develop/api-reference>, Accessed on: 19 Dec 2025.

[14] Python Software Foundation, "The Python Standard Library", <https://docs.python.org/3/library/>, Accessed on: 15 Dec 2025.

[15] SQLite Documentation, "SQLite Python Interface", <https://docs.python.org/3/library/sqlite3.html>, Accessed on: 18 Dec 2025.

[16] Pandas Documentation, "Intro to Data Structures: DataFrame", <https://pandas.pydata.org/docs/>, Accessed on: 18 Dec 2025.



## APPENDIX

### Appendix A: Source Code (Python & Streamlit)

The following is the complete source code for the **Pharmacy Management System** (main.py). It includes the library imports, database connection logic, and the functions for both the Admin and Customer interfaces.

Python

```
import streamlit as st

import pandas as pd

from PIL import Image

import random

import sqlite3

# -----

# DATABASE CONNECTION & MANAGEMENT

# -----

conn = sqlite3.connect("drug_data.db", check_same_thread=False)

c = conn.cursor()

# --- Customer Table Functions ---

def cust_create_table():

    c.execute("CREATE TABLE IF NOT EXISTS Customers(
```

```

C_Name VARCHAR(50) NOT NULL,

C_Password VARCHAR(50) NOT NULL,

C_Email VARCHAR(50) PRIMARY KEY NOT NULL,

C_State VARCHAR(50) NOT NULL,

C_Number VARCHAR(50) NOT NULL

)"""

```

```

def customer_add_data(Cname, Cpass, Cemail, Cstate, Cnumber):

```

```

    c.execute("INSERT INTO Customers (C_Name,C_Password,C_Email, C_State,
C_Number)

```

```

        VALUES(?,?,?,?,?)", (Cname, Cpass, Cemail, Cstate, Cnumber))

```

```

    conn.commit()

```

```

def customer_view_all_data():

```

```

    c.execute('SELECT * FROM Customers')

```

```

    return c.fetchall()

```

```

def customer_update(Cemail, Cnumber):

```

```

    c.execute(" UPDATE Customers SET C_Number = ? WHERE C_Email = ?", (Cnumber,
Cemail))

```

```

    conn.commit()

```

```
def customer_delete(Cemail):
```

```
    c.execute("DELETE FROM Customers WHERE C_Email = ?", (Cemail,))
```

```
    conn.commit()
```

```
# --- Drug Table Functions ---
```

```
def drug_create_table():
```

```
    c.execute("CREATE TABLE IF NOT EXISTS Drugs(
```

```
        D_Name VARCHAR(50) NOT NULL,
```

```
        D_ExpDate DATE NOT NULL,
```

```
        D_Use VARCHAR(50) NOT NULL,
```

```
        D_Qty INT NOT NULL,
```

```
        D_id INT PRIMARY KEY NOT NULL)
```

```
    ")
```

```
def drug_add_data(Dname, Dexpdate, Duse, Dqty, Did):
```

```
    c.execute("INSERT INTO Drugs (D_Name, D_Expdate, D_Use, D_Qty, D_id)
```

```
        VALUES(?,?,?,?,?)", (Dname, Dexpdate, Duse, Dqty, Did))
```

```
    conn.commit()
```

```

def drug_view_all_data():

    c.execute('SELECT * FROM Drugs')

    return c.fetchall()


def drug_update(Duse, Did):

    c.execute(" UPDATE Drugs SET D_Use = ? WHERE D_id = ?", (Duse, Did))

    conn.commit()


def drug_delete(Did):

    c.execute(" DELETE FROM Drugs WHERE D_id = ?", (Did,))

    conn.commit()


# --- Order Table Functions ---

def order_create_table():

    c.execute("CREATE TABLE IF NOT EXISTS Orders(

        O_Name VARCHAR(100) NOT NULL,

        O_Items VARCHAR(100) NOT NULL,

        O_Qty VARCHAR(100) NOT NULL,

        O_id VARCHAR(100) PRIMARY KEY NOT NULL)

    ")

```

```

def order_add_data(O_Name, O_Items, O_Qty, O_id):

    c.execute("INSERT INTO Orders (O_Name, O_Items, O_Qty, O_id)

                VALUES(?,?,?,?)", (O_Name, O_Items, O_Qty, O_id))

    conn.commit()


def order_view_data(customername):

    c.execute('SELECT * FROM ORDERS Where O_Name == ?', (customername,))

    return c.fetchall()


def order_view_all_data():

    c.execute('SELECT * FROM ORDERS')

    return c.fetchall()


# -----

# APPLICATION INTERFACE (FRONTEND)

# -----


def admin():

    st.title("Pharmacy Database Dashboard")

```

```
menu = ["Drugs", "Customers", "Orders", "About"]
```

```
choice = st.sidebar.selectbox("Menu", menu)
```

```
if choice == "Drugs":
```

```
    menu = ["Add", "View", "Update", "Delete"]
```

```
    choice = st.sidebar.selectbox("Menu", menu)
```

```
if choice == "Add":
```

```
    st.subheader("Add Drugs")
```

```
    col1, col2 = st.columns(2)
```

```
    with col1:
```

```
        drug_name = st.text_area("Enter the Drug Name")
```

```
        drug_expiry = st.date_input("Expiry Date of Drug (YYYY-MM-DD)")
```

```
        drug_mainuse = st.text_area("When to Use")
```

```
    with col2:
```

```
        drug_quantity = st.text_area("Enter the quantity")
```

```
        drug_id = st.text_area("Enter the Drug id (example:#D1)")
```

```
if st.button("Add Drug"):
```

```
    drug_add_data(drug_name, drug_expiry, drug_mainuse, drug_quantity, drug_id)
```

```

        st.success("Successfully Added Data")

elif choice == "View":

    st.subheader("Drug Details")

    drug_result = drug_view_all_data()

    with st.expander("View All Drug Data"):

        drug_clean_df = pd.DataFrame(drug_result, columns=["Name", "Expiry Date",
"Use", "Quantity", "ID"])

        st.dataframe(drug_clean_df)

elif choice == "Customers":

    st.subheader("Customer Details")

    cust_result = customer_view_all_data()

    with st.expander("View All Customer Data"):

        cust_clean_df = pd.DataFrame(cust_result, columns=["Name", "Password", "Email-
ID", "Area", "Number"])

        st.dataframe(cust_clean_df)

elif choice == "Orders":

    st.subheader("Order Details")

    order_result = order_view_all_data()

```

```

with st.expander("View All Order Data"):

    order_clean_df = pd.DataFrame(order_result, columns=["Name", "Items", "Qty",
"ID"])

    st.dataframe(order_clean_df)


def customer(username, password):

    if getauthenticate(username, password):

        st.title("Welcome to Pharmacy Store")

        st.subheader("Your Order Details")

        order_result = order_view_data(username)

        with st.expander("View Your Order History"):

            order_clean_df = pd.DataFrame(order_result, columns=["Name", "Items", "Qty",
"ID"])

            st.dataframe(order_clean_df)


        drug_result = drug_view_all_data()

        # Dynamic Product Display

        st.subheader("Drug: " + drug_result[0][0])

        img = Image.open('images/dolo650.jpg')

        st.image(img, width=100, caption="Rs. 15/-")

```



```

dolo650 = st.slider(label="Quantity", min_value=0, max_value=5, key=1)

st.info("When to USE: " + str(drug_result[0][2]))


if st.button(label="Buy now"):

    O_items = ""

    if int(dolo650) > 0: O_items += "Dolo-650,"

    O_Qty = str(dolo650)

    O_id = username + "#O" + str(random.randint(0, 1000000))

    order_add_data(username, O_items, O_Qty, O_id)


def getauthenticate(username, password):

    c.execute('SELECT C_Password FROM Customers WHERE C_Name = ?', (username,))

    cust_password = c.fetchall()

    if cust_password and cust_password[0][0] == password:

        return True

    else:

        return False


if __name__ == '__main__':

    drug_create_table()

```

```

cust_create_table()

order_create_table()


menu = ["Login", "SignUp", "Admin"]

choice = st.sidebar.selectbox("Menu", menu)


if choice == "Login":

    username = st.sidebar.text_input("User Name")

    password = st.sidebar.text_input("Password", type='password')

    if st.sidebar.checkbox(label="Login"):

        customer(username, password)


elif choice == "SignUp":

    st.subheader("Create New Account")

    cust_name = st.text_input("Name")

    cust_password = st.text_input("Password", type='password')

    cust_email = st.text_area("Email ID")

    cust_area = st.text_area("State")

    cust_number = st.text_area("Phone Number")

```

```
if st.button("Signup"):

    customer_add_data(cust_name, cust_password, cust_email, cust_area, cust_number)

    st.success("Account Created!")
```

```
elif choice == "Admin":

    username = st.sidebar.text_input("User Name")

    password = st.sidebar.text_input("Password", type='password')

    if username == 'admin' and password == 'admin':

        admin()
```

## Appendix B: Raw Data Samples

The following data represents the raw content extracted from the SQLite database file (drug\_data.db) during testing.

Table: Customers

| Name                | Password | Email                | State        | Number     |
|---------------------|----------|----------------------|--------------|------------|
| ----                | ----     | ----                 | ----         | ----       |
| Himanshu Sharma     | Himanshu | himanshu@gmail.com   | Chhattisgarh | 8529154560 |
| Ainesh Kumar        | Ainesh   | ainesh@gmail.com     | Maharashtra  | 7982658522 |
| Arshia Goel         | Arshia   | arshia@gmail.com     | Delhi        | 7896541652 |
| Varsha Gummagolmath | Varsha   | varsha45@outlook.com | Telangana    | 6985422328 |

Table: Orders

| Order ID                | Customer Name   | Items                      | Qty     |
|-------------------------|-----------------|----------------------------|---------|
| ----                    | ----            | ----                       | ----    |
| Himanshu Sharma#O415872 | Himanshu Sharma | Dolo-650, Strepsils, Vicks | 1, 3, 1 |
| Ainesh Kumar#O695796    | Ainesh Kumar    | Dolo-650, Strepsils, Vicks | 4, 1, 2 |
| Krishna Pandey#O675297  | Krishna Pandey  | Dolo-650, Strepsils, Vicks | 1, 5, 2 |
| Divyansh#O627428        | Divyansh        | Dolo-650, Strepsils, Vicks | 3, 2, 5 |

Table: Drugs

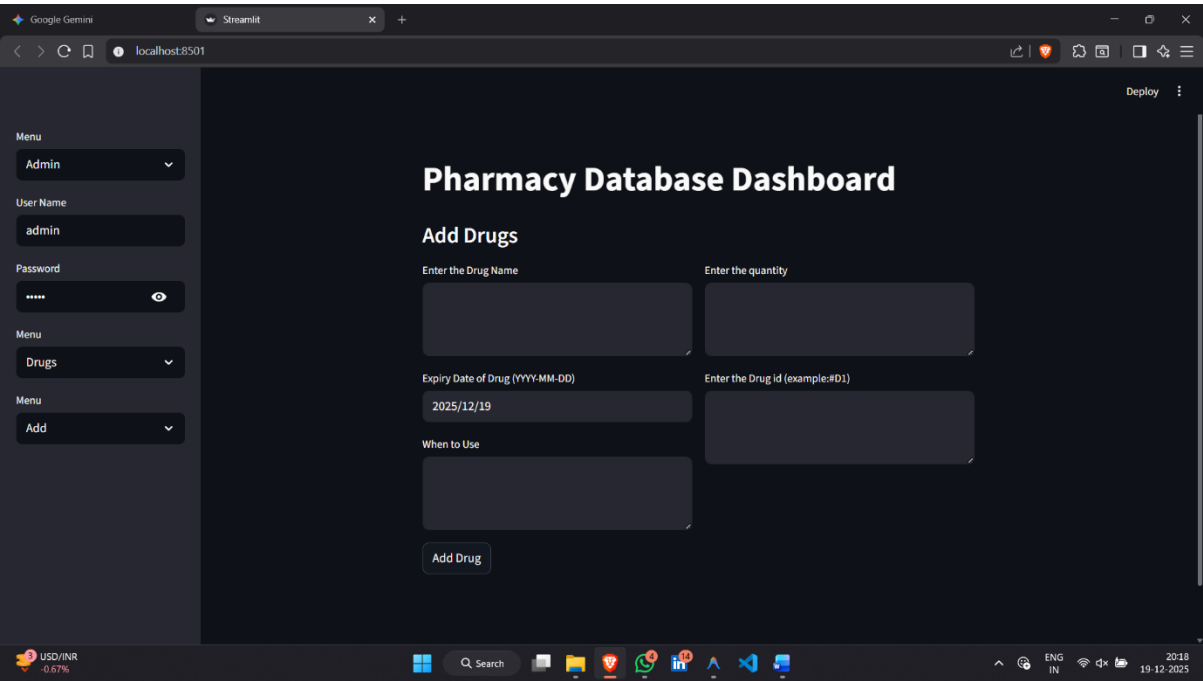
| Drug ID | Name     | Expiry Date | Use   | Quantity |
|---------|----------|-------------|-------|----------|
| ----    | ----     | ----        | ----  | ----     |
| #D1     | Dolo 650 | 2024-12-09  | Fever | 20       |

| #D2 | Strepsils | 2024-12-20 | Sore Throat | 50 |

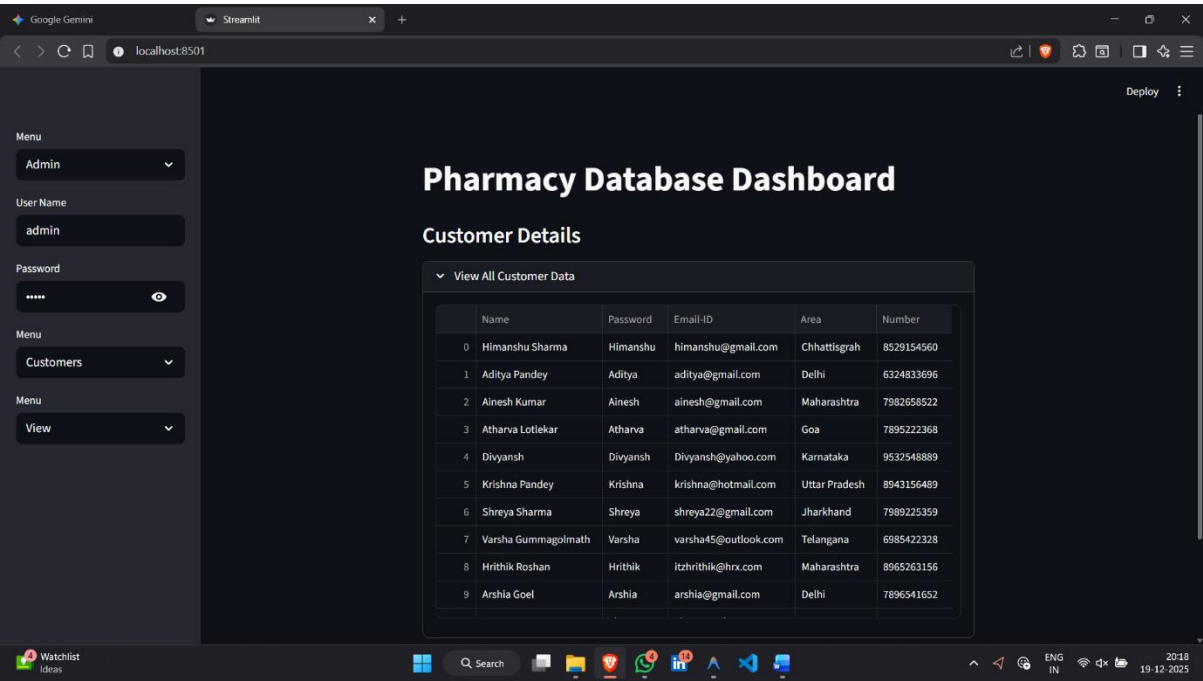
| #D3 | Vicks VaporRub | 2024-12-09 | Cold | 30 |

# SCREENSHOTS

## 1. Add Drugs



## 2. Customer Details



3. Order Details

Menu

Admin

User Name

admin

Password

\*\*\*\*\*

Menu

Orders

Menu

View

Pharmacy Database Dashboard

Order Details

View All Order Data

|   | Name            | Items                    | Qty   | ID                      |
|---|-----------------|--------------------------|-------|-------------------------|
| 0 | Himanshu Sharma | Dolo-650,Strepsils,      | 1,2,0 | Himanshu Sharma#O602700 |
| 1 | Himanshu Sharma | Dolo-650,Strepsils,Vicks | 1,2,1 | Himanshu Sharma#O476571 |
| 2 | Divyansh        | Dolo-650,Strepsils,Vicks | 3,2,5 | Divyansh#O627428        |
| 3 | Arshia Goel     | Dolo-650,Vicks           | 3,0,1 | Arshia Goel#O766239     |
| 4 | Krishna Pandey  | Dolo-650,Strepsils,Vicks | 1,5,2 | Krishna Pandey#O675297  |
| 5 | Ainesh Kumar    | Dolo-650,Strepsils,Vicks | 4,1,2 | Ainesh Kumar#O695796    |
| 6 | Himanshu Sharma | Dolo-650,Strepsils,Vicks | 1,3,1 | Himanshu Sharma#O415872 |
| 7 | ABC             | Dolo-650,Strepsils,Vicks | 2,3,1 | ABC#O362791             |

4. Drug Details

Menu

Admin

User Name

admin

Password

\*\*\*\*\*

Menu

Drugs

Menu

View

Pharmacy Database Dashboard

Drug Details

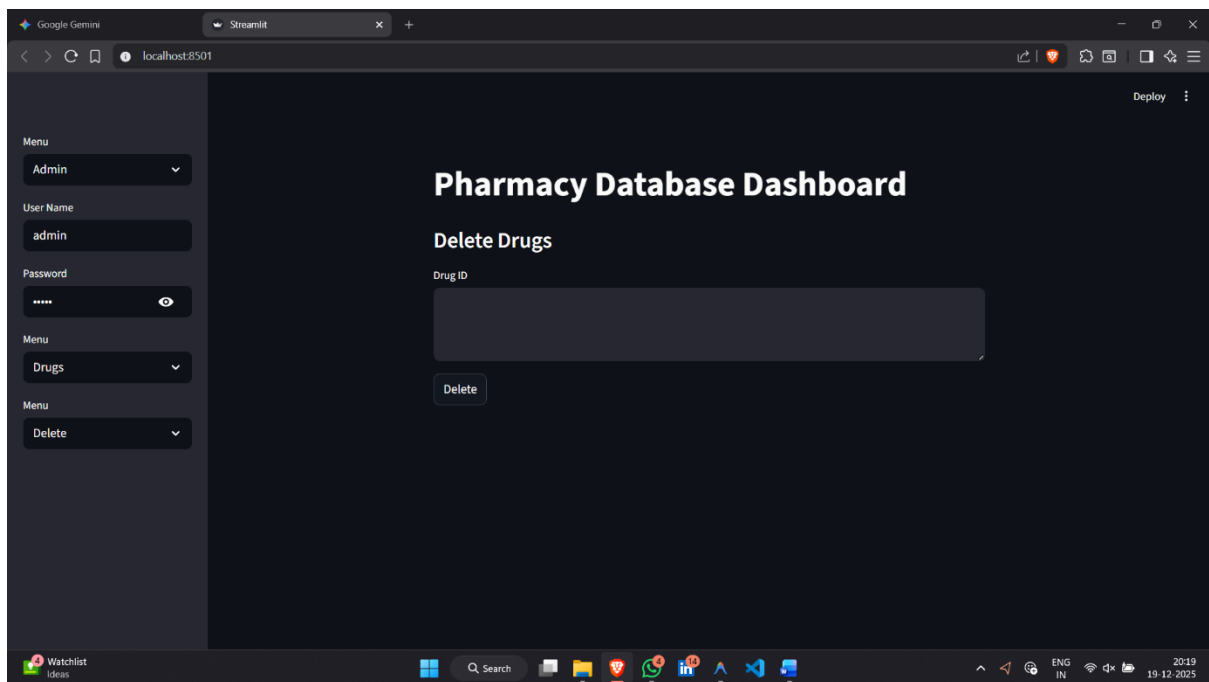
View All Drug Data

|   | Name           | Expiry Date | Use         | Quantity | ID  |
|---|----------------|-------------|-------------|----------|-----|
| 0 | Dolo 650       | 2024-12-09  | Fever       | 500      | #D1 |
| 1 | Strepsils      | 2024-12-20  | Sore Throat | 200      | #D2 |
| 2 | Vicks VaporRub | 2024-12-09  | Cold        | 200      | #D3 |

View Drug Quantity

|   | Name           | Quantity |
|---|----------------|----------|
| 0 | Dolo 650       | 500      |
| 1 | Strepsils      | 200      |
| 2 | Vicks VaporRub | 200      |

## 5. Delete Drugs



## 6. Login Page

